

## Atividade\_06 – Display LCD

### Título: Interfaceando um pedal de acelerador com Arduino (parte 2 – exibindo dados no display LCD)

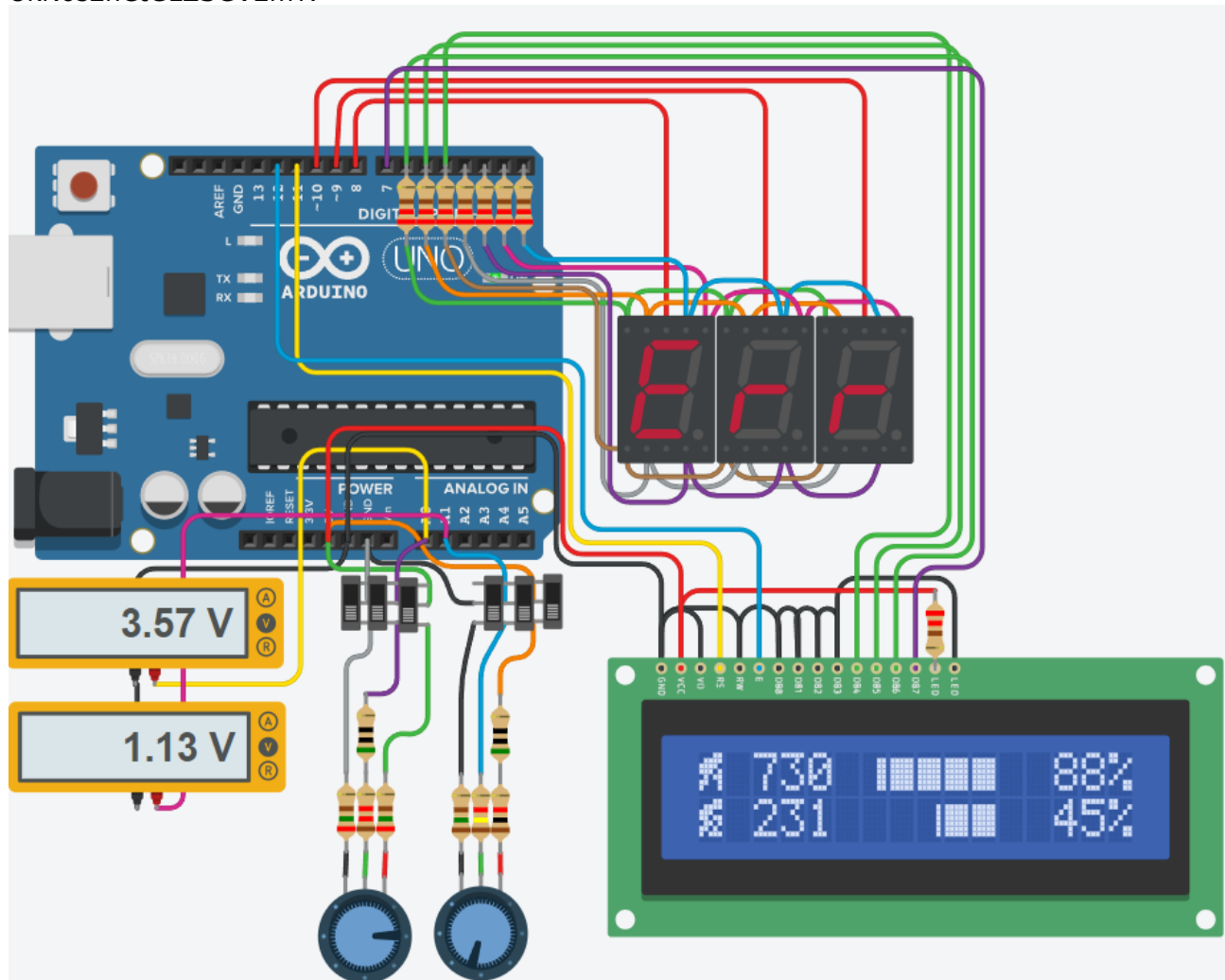
**Objetivos:** O objetivo é exibir os dados do pedal do acelerador da prática anterior no display de cristal líquido.

Nesta prática utilizaremos o **Tinkercad** para simular um circuito simples usando o microcontrolador Atmega328, utilizado nas placas Arduino UNO.

#### 1. Procedimentos:

1. Copie o projeto fornecido em

[https://www.tinkercad.com/things/4gqjovZVRMF?sharecode=CgXy80cym614CZFVCNC3d\\_U8BUkNos2nCtGLZSGV2mY](https://www.tinkercad.com/things/4gqjovZVRMF?sharecode=CgXy80cym614CZFVCNC3d_U8BUkNos2nCtGLZSGV2mY):



É possível ligar o LCD nos mesmos pinos do display de 7 segmentos pois o LCD só faz a leitura dos dados na transição do pino enable. O único cuidado necessário é desligar todos os displays de 7 segmentos no momento em que for enviar qualquer dado para o LCD.

2. Você deve exibir o valor bruto lido pelo ADC e o valor no intervalo percentual. O canal 0 do ADC deve ficar na linha de cima e o valor do canal 1 na linha de baixo.

O formato deve ser o seguinte:

|    |  |    |    |    |  |    |    |    |    |    |  |    |    |    |     |
|----|--|----|----|----|--|----|----|----|----|----|--|----|----|----|-----|
| S1 |  | R2 | R1 | R0 |  | OP | OP | OP | OP | OP |  | P2 | P1 | P0 | '%' |
|----|--|----|----|----|--|----|----|----|----|----|--|----|----|----|-----|

|    |  |    |    |    |  |    |    |    |    |    |  |    |    |    |     |
|----|--|----|----|----|--|----|----|----|----|----|--|----|----|----|-----|
| S2 |  | R2 | R1 | R0 |  | OP | OP | OP | OP | OP |  | P2 | P1 | P0 | '%' |
|----|--|----|----|----|--|----|----|----|----|----|--|----|----|----|-----|

Onde S1 e S2 são símbolos personalizados para indicar o potenciômetro 1 e 2. R2 a R0 são os valores brutos lidos pelo ADC, P2 a P0 são os valores de aceleração percentual. Note que no caso de percentuais como 34% ou 5%, os caracteres P2 e P2P1 não devem ser exibidos, mas o número continua alinhado à direita do display.

3. OP são uma escala em barras. Exibir as barras com o percentual de cada potenciômetro (criar 5 símbolos especiais para representar valores fracionários, usando 5 caracteres de 5 colunas, temos uma resolução de 4 pontos percentuais por coluna do display). Utilizar as barras da direita para a esquerda, ou seja, a barra fica alinhada a direita, como na imagem.

A taxa de atualização do LCD deve ser de **10Hz**, ou seja, você deve atualizar o display LCD somente **a cada 100ms**.

**RESTRIÇÃO DE PROJETO:** O circuito utilizado no projeto deve ser o fornecido, podendo as mudanças ocorrerem somente em relação ao software. Não devem haver delays (espera ociosa), exceto para o caso da simulação rodar mais rapidamente (ou rotinas do display que já possuem tais delays). Não utilizar bibliotecas como a LiquidCrystal ou afins.

4. Cole o código fonte do microcontrolador ao final deste arquivo. Anexe também o fonte em um arquivo separado.

**OBRIGATÓRIO PARA AVALIAÇÃO:** torne seu projeto compartilhado por link (não público) e cole o link para ele aqui:

[https://www.tinkercad.com/things/cAruqQa6gj4-copy-of-liquidcrystal-cf-2026-pratica-6-modelo?sharecode=xdOX9c7S\\_WOha5Mnc19PAg7JR-c-alPitBudU\\_KGJ50](https://www.tinkercad.com/things/cAruqQa6gj4-copy-of-liquidcrystal-cf-2026-pratica-6-modelo?sharecode=xdOX9c7S_WOha5Mnc19PAg7JR-c-alPitBudU_KGJ50)

O link também está na fonte enviado em paralelo

**ATENÇÃO:** Documente seu código. Cada linha/bloco deve deixar explícito o seu papel.

#### RÚBRICA:

Lógica da programação e funcionamento (desde que atendendo às restrições de projeto): 100%

- Exibição das barras proporcionais a aceleração com o uso de caracteres personalizados – 40% do total.

Valor desta atividade na média: 0.7

Nota extra: Os displays de 7 segmentos funcionais como na prática anterior: 0.1 na média

```
const int p1Min = 833;
const int p1Max = 190;
```

```
const int p2Min = 413;
const int p2Max = 87;
```

```
// controle do display
byte dig = 0;
```

```
unsigned long tDisp = 0;
unsigned long tLcd = 0;
```

```
// tabela dos numeros do display 7 segmentos
```

```

byte mapa[] = {
    0x40, 0x79, 0x24, 0x30, 0x19,
    0x12, 0x02, 0x78, 0x80, 0x18,
    0x06, 0x2F
};

// inicia mostrando "Err"
byte nums[3] = {10,11,11};

// valores lidos do adc
volatile int adc1 = 0;
volatile int adc2 = 0;

// valores usados no lcd
int b1 = 0;
int b2 = 0;

long pct1 = 0;
long pct2 = 0;

// caracteres personalizados do lcd
byte bar1[8] = {1,1,1,1,1,1,1,0};
byte bar2[8] = {3,3,3,3,3,3,3,0};
byte bar3[8] = {7,7,7,7,7,7,7,0};
byte bar4[8] = {15,15,15,15,15,15,15,0};
byte bar5[8] = {31,31,31,31,31,31,31,0};

byte s1[8] = {31,17,21,17,17,31,0,0};
byte s2[8] = {31,17,19,21,17,31,0,0};

// interrupção do adc
// alterna entre A0 e A1 automaticamente
ISR(ADC_vect)
{
    if (ADMUX & 1)
    {
        adc2 = ADC;

        // próxima leitura em A0
        ADMUX = (1 << REFS0);
    }
    else
    {
        adc1 = ADC;

        // próxima leitura em A1
        ADMUX = (1 << REFS0) | 1;
    }

    // inicia nova conversão
    ADCSRA |= (1 << ADSC);
}

// envia 4 bits para o lcd
void nibble(byte v)
{

```

```

// desliga display rapidamente
PORTB &= 0b11111000;

// envia parte alta
PORTD = (PORTD & 0x0F) | (v & 0xF0);

// pulso enable
PORTB |= (1 << PB4);
delayMicroseconds(2);

PORTB &= ~(1 << PB4);

delayMicroseconds(50);
}

```

```

// escreve comando ou dado no lcd
void lcd(byte v, bool rs)
{
    if (rs) PORTB |= (1 << PB3);
    else PORTB &= ~(1 << PB3);

    nibble(v);
    nibble(v << 4);
}

```

```

// cria caractere personalizado
void cria(byte pos, byte *d)
{
    lcd(0x40 | ((pos & 7) << 3), 0);

    for (int i = 0; i < 8; i++)
    {
        lcd(d[i], 1);
    }
}

```

```

// inicialização do lcd em 4 bits
void initLcd()
{
    delay(50);

    nibble(0x30);
    delay(5);

    nibble(0x30);
    delayMicroseconds(150);

    nibble(0x30);
    delayMicroseconds(150);

    nibble(0x20);

    lcd(0x28,0);
    lcd(0x0C,0);
    lcd(0x06,0);
    lcd(0x01,0);
}

```

```
delay(5);
```

```
// cria barras do gráfico
```

```
cria(0, bar1);
```

```
cria(1, bar2);
```

```
cria(2, bar3);
```

```
cria(3, bar4);
```

```
cria(4, bar5);
```

```
// simbolos S1 e S2
```

```
cria(5, s1);
```

```
cria(6, s2);
```

```
}
```

```
// atualiza uma linha do lcd
```

```
void linha(int l, int bruto, long pct, byte s)
```

```
{
```

```
    // seleciona linha
```

```
    lcd(l == 0 ? 0x80 : 0xC0, 0);
```

```
    // escreve simbolo
```

```
    lcd(s,1);
```

```
    lcd(' ',1);
```

```
    // limita valor bruto
```

```
    if (bruto > 999) bruto = 999;
```

```
    if (bruto < 0) bruto = 0;
```

```
    // escreve valor bruto
```

```
    if (bruto < 10)
```

```
    {
```

```
        lcd(' ',1);
```

```
        lcd(' ',1);
```

```
        lcd(bruto + '0',1);
```

```
    }
```

```
    else if (bruto < 100)
```

```
    {
```

```
        lcd(' ',1);
```

```
        lcd((bruto/10)+'0',1);
```

```
        lcd((bruto%10)+'0',1);
```

```
    }
```

```
    else
```

```
    {
```

```
        lcd((bruto/100)+'0',1);
```

```
        lcd(((bruto/10)%10)+'0',1);
```

```
        lcd((bruto%10)+'0',1);
```

```
    }
```

```
    lcd(' ',1);
```

```
// calcula quantidade da barra
```

```
int total = constrain(pct / 4, 0, 25);
```

```
// desenha barra
```

```
for (int i = 0; i < 5; i++)
```

```
{
```

```
    int v = total - ((4-i)*5);
```

```

    if (v >= 5) lcd(4,1);
    else if (v > 0) lcd(v-1,1);
    else lcd(' ',1);
}

```

```

lcd(' ',1);

```

```

// escreve porcentagem
if (pct < 10)
{
    lcd(' ',1);
    lcd(' ',1);
    lcd(pct+'0',1);
}
else if (pct < 100)
{
    lcd(' ',1);
    lcd((pct/10)+'0',1);
    lcd((pct%10)+'0',1);
}
else
{
    lcd('1',1);
    lcd('0',1);
    lcd('0',1);
}

lcd('%',1);
}

```

```

void setup()
{
    // portas do display e lcd
    DDRD = 0xFF;

    DDRB |= 0b00011111;

    // entradas analogicas
    DDRC &= ~(1 << PC0);
    DDRC &= ~(1 << PC1);

    initLcd();

    // referencia AVCC e canal A0 inicial
    ADMUX = (1 << REFS0);

    // habilita adc e interrupção
    ADCSRA =
    (1 << ADEN) |
    (1 << ADIF) |
    (1 << ADPS2) |
    (1 << ADPS1) |
    (1 << ADPS0);

    sei();

    // inicia primeira conversão

```

```

    ADCSRA |= (1 << ADSC);
}

void loop()
{
    // multiplexação display 7 segmentos
    if (millis() - tDisp >= 5)
    {
        tDisp = millis();

        dig++;

        if (dig > 2) dig = 0;

        PORTB = (PORTB & 0b11111000) | (1 << dig);

        PORTD = mapa[nums[dig]];
    }

    // copia valores do adc
    cli();

    int p1 = adc1;
    int p2 = adc2;

    sei();

    // converte para %
    pct1 = constrain(map(p1, p1Min, p1Max, 0, 100), 0, 100);
    pct2 = constrain(map(p2, p2Min, p2Max, 0, 100), 0, 100);

    b1 = p1;
    b2 = p2;

    // verifica leitura fora da faixa
    bool erro1 =
        (p1 < min(p1Min,p1Max)-10) ||
        (p1 > max(p1Min,p1Max)+10);

    bool erro2 =
        (p2 < min(p2Min,p2Max)-10) ||
        (p2 > max(p2Min,p2Max)+10);

    // verifica diferença entre sensores
    bool dif = abs(pct1 - pct2) > 10;

    // erro no sistema
    if (erro1 || erro2 || dif)
    {
        nums[0] = 10;
        nums[1] = 11;
        nums[2] = 11;
    }
    else
    {
        // média entre os sensores
        int media = (pct1 + pct2) / 2;
    }
}

```

```
    nums[0] = (media / 100) % 10;
    nums[1] = (media / 10) % 10;
    nums[2] = media % 10;

    if (media < 100) nums[0] = 0;
}

// atualização do lcd
if (millis() - tLcd >= 150)
{
    tLcd = millis();

    linha(0, b1, pct1, 5);
    linha(1, b2, pct2, 6);
}
}
```